

Fakturama Image-to-Cash Automation

Design document

Purpose: Part 1 submission - design only

Target: One supplied order image to saved Order and linked Invoice

Platform: Windows desktop, Fakturama, OCR and Microsoft UI Automation

Design principle: Verify each state transition before continuing

Scope note: This document explains the complete approach I would use for the task. The README records what was actually implemented, tested, and left incomplete.

1. Design summary

I approached the task as a sequence of small, verifiable actions instead of one long desktop macro. The order image is read and checked first. Only then does the program interact with Fakturama. After each important action, it reads the screen or control again to confirm that the expected change happened. If the source data or the application state is unclear, the program stops and asks for manual review rather than continuing with a guess.

2. Goals and design principles

- Use visible control properties and OCR results instead of relying on one set of fixed screen coordinates.
- Treat the Debtor and Product selectors opened from the Order as the main source of truth for existing records.
- Create a missing master record only after an exact match cannot be found, then return to the Order and select it again.
- Check the arithmetic before changing Fakturama and verify the values again after entry.
- Save screenshots and OCR output when a step cannot be verified so the failure can be investigated.

3. End-to-end architecture

Extract: OCR the supplied image and normalize dates, identifiers, addresses, payment data, item rows, currency, VAT, discounts, and totals.

Validate: Build a typed order model and reconcile calculated line, net, VAT, and gross totals with the source.

Create Order: Open a fresh Order, preserve the proposed number, set the reference/date/modes, and keep the editor open.

Resolve masters: Select or conditionally create the Debtor, payment method, VAT, and every Product using exact-match policies.

Complete and persist: Populate all line values, verify totals, save once, and prove the Order exists in Documents.

Create linked Invoice: Use the saved Order's follow-up action, copy-check the transaction, apply payment status, save, and verify both records.

4. Image extraction and validation

The extraction step keeps the original OCR text as well as the cleaned values used by the program. Labels and item rows are found from their position in the image, then each field is normalized separately. This includes dates, decimal separators, currency symbols, percentages, and identifiers. The order is rejected before UI automation if a required field is missing or if the totals do not reconcile.

Stage	Validation
Document fields	Require Order Date, External Reference, currency, customer identity, both addresses, and payment information.
Item rows	Require one SKU, description, positive quantity, unit net price, VAT, discount, and calculated line amount per source row.
Financials	Recalculate line net, VAT, net total, VAT total, and gross total using decimal arithmetic and two-decimal rounding.
Confidence	Escalate missing, conflicting, or multiply matched data for manual review; never silently choose between candidates.

5. Control discovery and grounding

Fakturama uses SWT controls, and several of them do not expose reliable names or positions through Windows automation. I therefore use semantic Win32 or UI Automation properties when they are available, limit every search to the active Fakturama window or dialog, and use OCR only for controls or table rows that cannot be identified reliably in another way.

Semantic lookup: Find labels, edits, combo boxes, buttons, toolbars, groups, and dialogs by role and visible text; require uniqueness inside the current context.

Structural relationships: Associate a field with the nearest compatible control on the same row, or locate repeated action icons relative to a verified section label.

OCR grounding: Read the current window image, constrain candidates to the expected region, and convert the matched image point using live window geometry.

Visual confirmation: After a physical click, confirm the expected dialog/editor appears or that the intended table row is visibly selected before pressing OK.

Coordinate handling: A coordinate is calculated from the current window or control rectangle. A click is considered successful only when the expected dialog, editor, or selected row is visible afterwards.

6. Master-data resolution

I keep the Order open while resolving Debtors, VAT records, payment methods, and Products. One exact match can be reused. More than one possible match stops the run because selecting the wrong accounting record would be worse than requiring manual input. If no match exists, the record can be created, saved once, and then selected again from the original Order to confirm that it was stored correctly.

- Debtor identity uses company, contact name, ZIP, city, and populated billing/delivery addresses.
- Payment methods require exact names and expected payment-code mappings before reuse or creation.
- VAT reuse requires exact name, percentage value, and standard-rate e-invoice code.
- Product identity uses the exact SKU; master gross price excludes the transaction-line discount.

7. State management, verification, and recovery

Before moving to the next step, the program checks both where it is and what changed. The active document number and customer reference are used to make sure the correct Order is being edited. Entered values are read back from the controls or from OCR. After saving, the program checks Data > Documents and matches several fields from the same row instead of trusting a single OCR token.

Transition	Success condition	Failure behavior
Open editor	Required labels and a new document control/number are present	Capture evidence and stop
Select master	Selector closes and populated transaction data or exact row is verified	Retry bounded input, then stop
Complete line	Qty, price, VAT, discount, and calculated line amount match	Do not save
Save Order/Invoice	Expected document row, reference, state, and total are persisted	Do not create downstream documents

8. Tradeoffs

- UI automation follows the required visible workflow and does not need a Fakturama API, but it is sensitive to focus, DPI, language, application version, and desktop state.
- OCR helps with SWT tables that expose little accessibility information, but OCR can misread text. Matching several fields reduces that risk.
- Exact matching protects accounting data from accidental reuse, although it can reject a valid record when the visible text is truncated or formatted differently.
- Keeping the Order open preserves context, but the automation must carefully reactivate the correct editor after working with master data.
- Screenshots are useful for debugging and submission evidence, but they must be reviewed for private or unrelated information before publication.

9. Testing strategy

Unit tests: Check decimal rounding, line calculations, totals, required fields, and payment-date rules.

OCR tests: Use clean, scaled, blurred, and ambiguous image crops for headers, SKUs, totals, and document numbers.

UI tests: Run against a disposable Fakturama workspace with both existing and missing Debtors, VAT records, payment methods, and Products.

Failure cases: Confirm that duplicates, missing controls, wrong active documents, and mismatched totals stop the workflow without saving.

End-to-end test: Record a complete Order-to-Invoice run and verify the final rows and payment state in Documents.

10. Production hardening

The first production improvement I would make is a resumable checkpoint based on the source reference. This would prevent a rerun from creating a duplicate after a failure that happened after saving. I would also make the OCR and locale settings configurable, test several document layouts and display configurations, and store structured audit logs. These changes would reduce the current dependence on one source image and one Windows/Fakturama setup.

Final design choice: Use semantic controls where possible, OCR where necessary, and verify every important result. If the program cannot prove that an accounting action succeeded, it should stop.